

Unit 14. Convex Programming, Support Vector Machines, Neural Networks

1 The Maximal Margin Classifier

Prerequisites §§P.3 and P.3b (convexity and separation; convex programming and duality).

This unit is outside the examination syllabus.

1.1 Margins

Throughout, the data are $(\mathbf{x}_i, y_i)$ with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$.

Definition 14.1 (Hyperplane and margins). For $\mathbf{w} \neq \mathbf{0}$ and $b \in \mathbb{R}$ the hyperplane is $\{\mathbf{x} : \mathbf{w}'\mathbf{x} + b = 0\}$ and the classifier is $\text{sign}(\mathbf{w}'\mathbf{x} + b)$. The **functional margin** of observation i is $\hat{\gamma}_i = y_i(\mathbf{w}'\mathbf{x}_i + b)$ and its **geometric margin** is $\gamma_i = \hat{\gamma}_i/|\mathbf{w}|$. The data are **separable** if some $(\mathbf{w}, b)$ has $\hat{\gamma}_i > 0$ for every i .

Proposition 14.2 (The geometric margin is a distance). γ_i is the signed Euclidean distance from $\mathbf{x}_i$ to the hyperplane, positive when $\mathbf{x}_i$ is classified correctly. It is unchanged when $(\mathbf{w}, b)$ is replaced by $(c\mathbf{w}, cb)$ for $c > 0$, while $\hat{\gamma}_i$ is multiplied by c .

Proof. The point of the hyperplane nearest $\mathbf{x}_i$ is $\mathbf{x}_i - t\mathbf{w}$ with $\mathbf{w}'(\mathbf{x}_i - t\mathbf{w}) + b = 0$, so $t = (\mathbf{w}'\mathbf{x}_i + b)/|\mathbf{w}|^2$ and the distance is $|t| |\mathbf{w}| = |\mathbf{w}'\mathbf{x}_i + b|/|\mathbf{w}|$. Multiplying by y_i attaches the sign. The scaling claims are immediate, $|c\mathbf{w}| = c|\mathbf{w}|$ for $c > 0$. $\square$

Definition 14.3 (Maximal margin problem). Among separating hyperplanes, maximise the smallest geometric margin:

$$\max_{\mathbf{w} \neq \mathbf{0}, b} \min_i \frac{y_i(\mathbf{w}'\mathbf{x}_i + b)}{|\mathbf{w}|}. \quad (1)$$

Theorem 14.4 (Canonical form). Suppose the data are separable. Then (1) has the same solutions, up to the positive scaling of Proposition 14.2, as the convex programme

$$\min_{\mathbf{w}, b} \frac{1}{2}|\mathbf{w}|^2 \quad \text{subject to} \quad y_i(\mathbf{w}'\mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n, \quad (2)$$

and the maximised margin is $1/|\mathbf{w}^*|$, so the two margin boundaries are $2/|\mathbf{w}^*|$ apart.

Proof. By Proposition 14.2 the objective of (1) is invariant under $(\mathbf{w}, b) \mapsto (c\mathbf{w}, cb)$, $c > 0$, so the maximisation may be restricted to any one representative per ray. Choose the representative with $\min_i \hat{\gamma}_i = 1$, possible because the data are separable and $\min_i \hat{\gamma}_i > 0$ can be scaled to 1. On that set the objective is $1/|\mathbf{w}|$, so maximising it is minimising $|\mathbf{w}|$, equivalently $\frac{1}{2}|\mathbf{w}|^2$, subject to $\min_i \hat{\gamma}_i \geq 1$, which is (2).

The constraint $\min_i \hat{\gamma}_i \geq 1$ is binding at the optimum, since otherwise $(\mathbf{w}, b)$ could be divided by $\min_i \hat{\gamma}_i > 1$, reducing the objective while staying feasible. Hence $\min_i \gamma_i = 1/|\mathbf{w}^*|$ and the slab between the two margin boundaries has width twice that. $\square$

Theorem 14.5 (Existence and uniqueness). If the data are separable, (2) has exactly one solution $(\mathbf{w}^*, b^*)$.

Proof. The feasible set is an intersection of closed half-spaces, hence closed and convex, and it is non-empty by separability and Theorem 14.4. The objective $\frac{1}{2}|\mathbf{w}|^2$ is continuous and coercive in $\mathbf{w}$ on that set, and b is confined to a bounded range once $\mathbf{w}$ is bounded, because the constraints for one index of each sign give $-1 - \mathbf{w}'\mathbf{x}_i \leq b \leq \mathbf{w}'\mathbf{x}_j - 1$ for suitable i, j . So the minimisation may

be taken over a compact set and the infimum is attained (prerequisites, coercive minimisation). For uniqueness, $\mathbf{w} \mapsto \frac{1}{2}|\mathbf{w}|^2$ is strictly convex, so two solutions must share the same $\mathbf{w}^*$; and given $\mathbf{w}^*$ the feasible b form an interval on which the objective is constant, with the binding constraint of Theorem 14.4 forcing b to an endpoint at each end, hence to a single point. $\square$

Remark. Nothing in the argument mentions a probability model. The maximal margin classifier is a geometric object defined by an optimisation problem, and (2) is exactly the form the prerequisites' duality theory takes as input: a convex objective and affine constraints.

2 The Dual, Support Vectors and Kernels

Prerequisites §P.3b.

2.1 The dual problem

Theorem 14.6 (The dual of the maximal margin problem). For separable data the Lagrangian

dual of (2) is

$$\max_{\alpha \geq 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j \quad \text{subject to} \quad \sum_i \alpha_i y_i = 0, \quad (3)$$

strong duality holds, and at the optimum

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i, \quad |\mathbf{w}^*|^2 = \sum_i \alpha_i^*. \quad (4)$$

Proof. The Lagrangian of (2) with multipliers $\alpha_i \geq 0$ is

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} |\mathbf{w}|^2 - \sum_i \alpha_i \{y_i (\mathbf{w}' \mathbf{x}_i + b) - 1\},$$

which is convex in $(\mathbf{w}, b)$, so its infimum is where the gradient vanishes: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ from $\partial/\partial \mathbf{w}$, and $\sum_i \alpha_i y_i = 0$ from $\partial/\partial b$. If $\sum_i \alpha_i y_i \neq 0$ the infimum over b is $-\infty$, so the dual function is

finite only on that hyperplane. Substituting,

$$\inf_{\mathbf{w}, b} L = \frac{1}{2} \left| \sum_i \alpha_i y_i \mathbf{x}_i \right|^2 - \left| \sum_i \alpha_i y_i \mathbf{x}_i \right|^2 + \sum_i \alpha_i = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j,$$

which is the objective of (3). Maximising the dual function is the dual problem by the prerequisites' definition.

Strong duality: the constraints of (2) are affine and the problem is feasible, so Slater's condition holds in the form the prerequisites prove for affine constraints, and the two optimal values agree.

The first identity of (4) is the stationarity condition just derived. For the second, strong duality equates the primal value $\frac{1}{2} |\mathbf{w}^*|^2$ with the dual value $\sum_i \alpha_i^* - \frac{1}{2} |\mathbf{w}^*|^2$, using $\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j = |\mathbf{w}^*|^2$; rearranging gives $\sum_i \alpha_i^* = |\mathbf{w}^*|^2$. $\square$

Corollary 14.7 (Support vectors). At the optimum, complementary slackness gives

$$\alpha_i^* \{y_i(\mathbf{w}^{*'} \mathbf{x}_i + b^*) - 1\} = 0 \quad \text{for every } i. \quad (5)$$

So $\alpha_i^* > 0$ only for observations with functional margin exactly 1, that is lying on a margin

boundary; these are the **support vectors**. By Theorem 14.6, $\mathbf{w}^*$ is a combination of the support vectors alone, and the classifier

$$\text{sign}\left(\sum_{i \in \text{SV}} \alpha_i^* y_i \mathbf{x}_i' \mathbf{x} + b^*\right) \quad (6)$$

does not depend on the other observations at all. The margin width is $2/\sqrt{\sum_i \alpha_i^*}$.

Proof. (5) is the complementary slackness condition of the prerequisites' Karush–Kuhn–Tucker theorem, whose hypotheses hold here by the strong duality just established. If the constraint is slack then $\alpha_i^* = 0$, so only tight constraints carry weight; substituting (4) into $\mathbf{w}'\mathbf{x} + b$ gives (6), and the margin statement combines Theorem 14.4 with the second half of (4). $\square$

Remark. That the solution depends on a handful of observations is the substantive content, and it is not a design choice but a consequence of complementary slackness. Moving a point that is not a support vector, so long as it stays outside the slab, does not change the classifier at all. Contrast least squares, where every observation enters $\hat{\boldsymbol{\beta}}$ through $\mathbf{X}'\mathbf{Y}$.

2.2 Non-separable data

Definition 14.8 (Soft margin). For $C > 0$,

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} |\mathbf{w}|^2 + C \sum_i \xi_i \quad \text{subject to} \quad y_i(\mathbf{w}'\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (7)$$

Theorem 14.9 (Soft-margin dual and the box constraint). The dual of (7) is (3) with the single change that α is confined to the box $0 \leq \alpha_i \leq C$. Complementary slackness gives

$$\alpha_i = 0 \Rightarrow \hat{\gamma}_i \geq 1, \quad 0 < \alpha_i < C \Rightarrow \hat{\gamma}_i = 1, \quad \alpha_i = C \Rightarrow \hat{\gamma}_i \leq 1. \quad (8)$$

Proof. With multipliers $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$, the Lagrangian is $\frac{1}{2} |\mathbf{w}|^2 + C \sum_i \xi_i - \sum_i \alpha_i \{y_i(\mathbf{w}'\mathbf{x}_i + b) - 1 + \xi_i\} - \sum_i \mu_i \xi_i$. Stationarity in $\mathbf{w}$ and b gives the same two conditions as before, and stationarity in ξ_i gives $C - \alpha_i - \mu_i = 0$. Since $\mu_i \geq 0$ this is $\alpha_i \leq C$, and substituting removes ξ from the dual function, leaving the objective of (3).

For (8): $\alpha_i < C$ forces $\mu_i > 0$, hence $\xi_i = 0$ by complementary slackness on the second constraint, hence $\hat{\gamma}_i \geq 1$; and $\alpha_i > 0$ forces the first constraint tight, $\hat{\gamma}_i = 1 - \xi_i \leq 1$. Combining the two

gives the three cases. □

Proposition 14.10 (Hinge loss). Problem (7) is equivalent to the unconstrained

$$\min_{\mathbf{w}, b} \sum_{i=1}^n \max\{0, 1 - y_i(\mathbf{w}'\mathbf{x}_i + b)\} + \frac{1}{2C}|\mathbf{w}|^2, \quad (9)$$

a convex penalised loss of the shape Unit 6 studied, with the hinge in place of squared error and $1/(2C)$ in place of λ .

Proof. For fixed $(\mathbf{w}, b)$ the constraints of (7) say $\xi_i \geq \max\{0, 1 - \hat{\gamma}_i\}$, and the objective increases in ξ_i , so the optimum takes ξ_i equal to that lower bound. Substituting and dividing by C gives (9). Convexity: a maximum of two affine functions is convex, and $|\mathbf{w}|^2$ is convex. □

Remark. So C is a regularisation parameter read backwards. Large C penalises violations heavily, gives a narrow margin and few support vectors; small C gives a wide margin, many support vectors at the bound, and a smaller $|\mathbf{w}|$.

2.3 Kernels

Definition 14.11 (Kernel). A **kernel** is a symmetric $K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ that is **positive semi-definite**: for every finite set $\mathbf{x}_1, \dots, \mathbf{x}_n$ the Gram matrix $\mathbf{K} = [K(\mathbf{x}_i, \mathbf{x}_j)]$ is positive semi-definite.

Proposition 14.12 (A feature map gives a kernel, and makes the dual concave). If $K(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})' \varphi(\mathbf{z})$ for some $\varphi : \mathbb{R}^p \rightarrow \mathbb{R}^d$ then K is a kernel. For any kernel, the dual objective of (3) with $\mathbf{x}_i' \mathbf{x}_j$ replaced by $K(\mathbf{x}_i, \mathbf{x}_j)$ is concave in $\boldsymbol{\alpha}$, so (3) remains a convex programme and the prerequisites' duality theory still applies.

Proof. For a feature map, $\mathbf{c}' \mathbf{K} \mathbf{c} = \sum_{i,j} c_i c_j \varphi(\mathbf{x}_i)' \varphi(\mathbf{x}_j) = \left| \sum_i c_i \varphi(\mathbf{x}_i) \right|^2 \geq 0$. For concavity, write the quadratic term as $\boldsymbol{\alpha}' \mathbf{Q} \boldsymbol{\alpha}$ with $\mathbf{Q} = \text{diag}(\mathbf{y}) \mathbf{K} \text{diag}(\mathbf{y})$. For any $\mathbf{c}$, $\mathbf{c}' \mathbf{Q} \mathbf{c} = \mathbf{d}' \mathbf{K} \mathbf{d}$ with $d_i = y_i c_i$, which is non-negative because $\mathbf{K}$ is positive semi-definite. So $\mathbf{Q}$ is positive semi-definite, $-\frac{1}{2} \boldsymbol{\alpha}' \mathbf{Q} \boldsymbol{\alpha}$ is concave, and adding the linear term $\sum_i \alpha_i$ preserves concavity. $\square$

Example 14.1 (The quadratic kernel). For $p = 2$, $K(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}' \mathbf{z})^2$ equals $\varphi(\mathbf{x})' \varphi(\mathbf{z})$ with

$$\varphi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)',$$

as expanding $(1 + x_1 z_1 + x_2 z_2)^2$ shows. Fitting (3) with this K is fitting a linear machine in the six features without ever forming them, and §4.3 verifies the two give the same α .

Remark. The dual (3) and the classifier (6) involve the data only through inner products, which makes the substitution legitimate. For the radial kernel $K(\mathbf{x}, \mathbf{z}) = \exp(-\sigma|\mathbf{x} - \mathbf{z}|^2)$ the corresponding φ is infinite-dimensional, so the primal (2) cannot be written down at all while the dual is an ordinary finite quadratic programme in n variables, so the dual is the object one solves.

3 Neural Networks

Prerequisites §P.11 (matrix differentiation and the chain rule).

3.1 The multilayer perceptron

Definition 14.13 (One hidden layer). With activation $g : \mathbb{R} \rightarrow \mathbb{R}$ applied coordinatewise, weights $\mathbf{W}_1 \in \mathbb{R}^{H \times p}$, $\mathbf{w}_2 \in \mathbb{R}^H$ and offsets $\mathbf{c}_1 \in \mathbb{R}^H$, $c_2 \in \mathbb{R}$, the network is

$$\mathbf{z}(\mathbf{x}) = \mathbf{W}_1 \mathbf{x} + \mathbf{c}_1, \quad \mathbf{a}(\mathbf{x}) = g(\mathbf{z}(\mathbf{x})), \quad f(\mathbf{x}) = \mathbf{w}_2' \mathbf{a}(\mathbf{x}) + c_2. \quad (10)$$

The parameters are $\boldsymbol{\theta} = (\mathbf{W}_1, \mathbf{c}_1, \mathbf{w}_2, c_2)$, of dimension $Hp + 2H + 1$, and the fit minimises $L(\boldsymbol{\theta}) = \frac{1}{2n} \sum_i \{y_i - f(\mathbf{x}_i)\}^2$.

Remark. With H arbitrary and g the identity, f is linear in $\mathbf{x}$ and (10) is the model of Unit 2 with a redundant parametrisation: nothing is gained, and the composition of linear maps is linear however many layers are stacked. The activation is the whole of the difference.

3.2 Backpropagation

Theorem 14.14 (The gradient). Write $\mathbf{Z} = \mathbf{XW}'_1 + \mathbf{1c}'_1$ and $\mathbf{A} = g(\mathbf{Z})$, both $n \times H$, and $\mathbf{f} = \mathbf{Aw}_2 + c_2\mathbf{1}$. Put $\mathbf{d}_2 = (\mathbf{f} - \mathbf{y})/n \in \mathbb{R}^n$ and

$$\mathbf{D}_1 = (\mathbf{d}_2 \mathbf{w}'_2) \odot g'(\mathbf{Z}) \in \mathbb{R}^{n \times H}, \quad (11)$$

$\odot$ being entrywise multiplication. Then

$$\frac{\partial L}{\partial \mathbf{w}_2} = \mathbf{A}' \mathbf{d}_2, \quad \frac{\partial L}{\partial c_2} = \mathbf{1}' \mathbf{d}_2, \quad \frac{\partial L}{\partial \mathbf{W}_1} = \mathbf{D}'_1 \mathbf{X}, \quad \frac{\partial L}{\partial \mathbf{c}_1} = \mathbf{D}'_1 \mathbf{1}. \quad (12)$$

Proof. $L = \frac{1}{2n}|\mathbf{y} - \mathbf{f}|^2$, so $\partial L/\partial \mathbf{f} = (\mathbf{f} - \mathbf{y})/n = \mathbf{d}_2$ by the prerequisites' rule for the differential of a quadratic form.

Since $\mathbf{f} = \mathbf{A}\mathbf{w}_2 + c_2\mathbf{1}$ with $\mathbf{A}$ not depending on $\mathbf{w}_2$ or c_2 , the chain rule (prerequisites) gives $\partial L/\partial \mathbf{w}_2 = \mathbf{A}'\mathbf{d}_2$ and $\partial L/\partial c_2 = \mathbf{1}'\mathbf{d}_2$, which are the first two identities.

For the first layer, f_i depends on z_{ih} only through $a_{ih} = g(z_{ih})$ and $\partial f_i/\partial a_{ih} = w_{2h}$, so $\partial L/\partial z_{ih} = d_{2i}w_{2h}g'(z_{ih})$, which is entry (i, h) of (11). Then $z_{ih} = \mathbf{w}'_{1h}\mathbf{x}_i + c_{1h}$ gives $\partial z_{ih}/\partial w_{1hj} = x_{ij}$ and $\partial z_{ih}/\partial c_{1h} = 1$, so summing over i ,

$$\frac{\partial L}{\partial w_{1hj}} = \sum_i D_{1,ih}x_{ij}, \quad \frac{\partial L}{\partial c_{1h}} = \sum_i D_{1,ih},$$

which are the last two identities in matrix form. □

Remark. The name records the shape of the computation: $\mathbf{d}_2$ is formed at the output and pushed back through $\mathbf{w}_2$ and g' to give $\mathbf{D}_1$, and each layer's two derivatives are then one matrix product. The cost is that of one forward pass, whatever the number of parameters, so (12) and not a difference quotient is what is computed. All twenty-six partial derivatives of a small network agree

with central differences to 10^{-10} .

3.3 What one hidden layer can represent

Theorem 14.15 (Rectified units and piecewise linearity). Let $p = 1$ and $g(u) = \max(0, u)$. Then

- (i) every network (10) with H units is continuous and piecewise linear with breakpoints among the H points $-c_{1h}/w_{1h}$, hence has at most $H + 1$ linear pieces;
- (ii) conversely every continuous piecewise-linear f with breakpoints $t_1 < \dots < t_H$ is exactly representable by such a network with H units.

Proof. (i) Each term $w_{2h} \max(0, w_{1h}x + c_{1h})$ is continuous, is affine on each side of $-c_{1h}/w_{1h}$, and has no other kink; a finite sum of such functions is continuous and affine on each interval between consecutive kinks. With H kinks the line is cut into at most $H + 1$ intervals.

(ii) Let f have slope s_0 on $(-\infty, t_1)$ and slope s_h on (t_h, t_{h+1}) . Put $w_{1h} = 1$, $c_{1h} = -t_h$ and $w_{2h} = s_h - s_{h-1}$ for $h = 1, \dots, H$. The candidate $\tilde{f}(x) = c_2 + s_0x + \sum_h w_{2h} \max(0, x - t_h)$ has slope s_0 below t_1 and gains $s_h - s_{h-1}$ at each t_h , hence has the slopes of f everywhere; choosing

c_2 to match f at one point makes $\tilde{f} = f$, two continuous functions with equal derivatives on a connected set differing by a constant. The linear term s_0x is absorbed by one extra unit or by allowing an identity connection. $\square$

Remark. So a one-dimensional rectified network is precisely a piecewise-linear regression with free knots, and H counts the knots. That is a modest class, and the reason such networks are useful is not expressiveness at fixed H but that H may be taken large and the knots are fitted rather than chosen. Compare Unit 11: a tree is piecewise **constant** with knots fitted greedily one at a time.

3.4 Why the optimisation is different

Proposition 14.16 (Permutation symmetry). Let P be a permutation of $\{1, \dots, H\}$ and let $\boldsymbol{\theta}^P$ permute the rows of $\mathbf{W}_1$, the entries of $\mathbf{c}_1$ and the entries of $\mathbf{w}_2$ by P . Then f is unchanged, so $L(\boldsymbol{\theta}^P) = L(\boldsymbol{\theta})$. Consequently every minimiser with distinct hidden units belongs to an orbit of $H!$ minimisers, and L is not convex whenever $H \geq 2$.

Proof. $\mathbf{w}'_2 g(\mathbf{W}_1 \mathbf{x} + \mathbf{c}_1) = \sum_h w_{2h} g(\mathbf{w}'_{1h} \mathbf{x} + c_{1h})$ is a sum, unchanged by reordering its terms. For non-convexity, take two distinct minimisers $\boldsymbol{\theta}$ and $\boldsymbol{\theta}^P$ in one orbit; a convex function attaining its minimum at two points attains it on the segment between them, and the midpoint of $\boldsymbol{\theta}$ and $\boldsymbol{\theta}^P$ averages two different hidden units into one, generally giving a different and larger L . $\square$

Remark. The support vector machine of (2) is a convex programme, so its solution is unique (Theorem 14.5), the duality theory of the prerequisites applies exactly, and any algorithm that stops at a stationary point has found the answer. Nothing of this survives for (10): there are at least $H!$ global minima, no duality, and gradient descent returns whatever basin it started in.

Definition 14.17 (Gradient descent and its stochastic form). Gradient descent iterates $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla L(\boldsymbol{\theta})$ with step size $\eta > 0$. The **stochastic** form replaces ∇L by the same expression computed on a random subset of the observations, which is unbiased for ∇L when the subset is drawn uniformly.

Proposition 14.18 (Unbiasedness). Let $\mathcal{B}$ be a uniformly random subset of $\{1, \dots, n\}$ of size m and $L_{\mathcal{B}}(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i \in \mathcal{B}} \{y_i - f(\mathbf{x}_i)\}^2$. Then $\mathbf{E}[\nabla L_{\mathcal{B}}(\boldsymbol{\theta})] = \nabla L(\boldsymbol{\theta})$ for every fixed $\boldsymbol{\theta}$.

Proof. $\nabla L_{\mathcal{B}} = \frac{1}{m} \sum_{i \in \mathcal{B}} \mathbf{g}_i$ with $\mathbf{g}_i$ the gradient of the i th term, and each i belongs to $\mathcal{B}$ with probability m/n , so the expectation is $\frac{1}{m} \cdot \frac{m}{n} \sum_i \mathbf{g}_i = \nabla L$. $\square$

Remark. Unbiasedness of the step is all that is claimed. It is not a convergence theorem: for a non-convex L none is available at this level, and the practice of restarting, of shrinking η , and of stopping on a validation set is a response to exactly that absence.

4 Computation: Formulas Against Package Output

Prerequisites §P.3b; this unit's §§1–3.

The quadratic programmes of §2 are solved directly with `quadprog`, which minimises $\frac{1}{2} \mathbf{d}' \mathbf{D} \mathbf{d} - \mathbf{d}'_0 \mathbf{d}$ subject to $\mathbf{A}' \mathbf{d} \geq \beta_0$, and the results are checked against `e1071`.

```
1 suppressMessages({library(quadprog); library(e1071)})
2 set.seed(3); n <- 40
3 X <- matrix(rnorm(n*2), n, 2)
4 y <- ifelse(X[, 1] + X[, 2] > 0.6, 1, -1)
5 X[y == 1, ] <- X[y == 1, ] + 0.9
6 dualfit <- function(K, y, C) { m <- length(y)
```

```

7 D <- (y %o% y)*K + diag(1e-8, m)
8 s <- solve.QP(D, rep(1, m), cbind(y, diag(m), -diag(m)),
9               c(0, rep(0, m), rep(-C, m)), meq = 1)
10 pmax(s$solution, 0) }
11 c(n = n, positives = sum(y == 1), separable = TRUE)

```

```

##           n positives separable
##          40           14           1

```

4.1 The dual and the support vectors (§2.1)

```

1 K1 <- X %*% t(X); Cbig <- 1e6
2 al <- dualfit(K1, y, Cbig)
3 w <- drop(t(X) %*% (al*y)); sv <- which(al > 1e-6)
4 b0 <- mean(y[sv] - X[sv, ] %*% w)
5 c(support_vectors = length(sv), of_n = n,
6   constraint_sum_alpha_y = sum(al*y),
7   norm_w_squared = sum(w^2), sum_alpha = sum(al),
8   margin_from_w = 2/sqrt(sum(w^2)),
9   margin_from_alpha = 2/sqrt(sum(al)))

```

```

##           support_vectors           of_n

```

```
##          3.0000000000          40.0000000000
## constraint_sum_alpha_y      norm_w_squared
##        -0.0000001979          2.1821910072
##          sum_alpha          margin_from_w
##        2.1821910036          1.3538907304
##      margin_from_alpha
##        1.3538907316
```

The two expressions for the margin agree because $|\mathbf{w}^*|^2 = \sum_i \alpha_i^*$, which is the half of (4) that strong duality supplies. Complementary slackness (5) says the functional margin is exactly 1 wherever $\alpha_i > 0$, and at least 1 everywhere.

```
1 fm <- drop(y*(X %*% w + b0))
2 c(max_deviation_from_one_at_SV = max(abs(fm[sv] - 1)),
3   smallest_functional_margin = min(fm),
4   largest_alpha_off_the_margin = max(al[abs(fm - 1) > 1e-4]))
```

```
## max_deviation_from_one_at_SV      smallest_functional_margin
##          0.00000021356          0.99999994213
## largest_alpha_off_the_margin
##          0.00000000485
```

e1071 orders the two classes the other way, so its $(\mathbf{w}, b)$ is the negative of the one above and the hyperplane is the same; the row of sums is what should vanish, and does, to the four decimal places its own solver works to.

```
1 m1 <- svm(X, factor(y), kernel = "linear", cost = Cbig, scale = FALSE)
2 wp <- drop(t(m1$coefs) %*% m1$SV)
3 rbind(manual = c(w = w, b = b0), e1071 = c(wp, -m1$rho),
4       sum = c(w + wp, b0 - m1$rho))
```

```
##           w1           w2           b
## manual  1.0655414  1.0231386 -1.5651149
## e1071   -1.0656645 -1.0226707  1.5647157
## sum     -0.0001231  0.0004679 -0.0003993
```

```
1 c(support_vectors_manual = length(sv), e1071 = nrow(m1$SV))
```

```
## support_vectors_manual           e1071
##                   3                   3
```

4.2 The soft margin (§2.2)

The data are made non-separable by flipping six labels.

```

1 set.seed(5); n2 <- 60
2 X2 <- matrix(rnorm(n2*2), n2, 2)
3 y2 <- ifelse(X2[, 1] + X2[, 2] > 0, 1, -1)
4 y2[sample(n2, 6)] <- -y2[sample(n2, 6)]
5 K2 <- X2 %*% t(X2)
6 t(sapply(c(0.1, 1, 10), function(C) {
7   a <- dualfit(K2, y2, C); ww <- drop(t(X2) %*% (a*y2))
8   free <- a > 1e-7 & a < C - 1e-7
9   bb <- if (any(free)) mean(y2[free] - X2[free, , drop = FALSE] %*% ww) else 0
10  g <- drop(y2*(X2 %*% ww + bb))
11  pk <- svm(X2, factor(y2), kernel = "linear", cost = C, scale = FALSE)
12  c(C = C, norm_w = sqrt(sum(ww^2)),
13    norm_w_e1071 = sqrt(sum(drop(t(pk$coefs) %*% pk$SV)^2)),
14    margin_width = 2/sqrt(sum(ww^2)),
15    support_vectors = sum(a > 1e-7), at_the_bound = sum(a > C - 1e-7),
16    max_margin_of_free_SV = if (any(free)) max(abs(g[free] - 1)) else NA) })))

```

##		C	norm_w	norm_w_e1071	margin_width	support_vectors
##	[1,]	0.1	1.178	1.177	1.6975	35
##	[2,]	1.0	2.270	2.271	0.8811	22

```
## [3,] 10.0  2.778      2.778      0.7200      18
##      at_the_bound max_margin_of_free_SV
## [1,]      34      0.0000000000
## [2,]      19      0.0000006426
## [3,]      15      0.0000007412
```

$|\mathbf{w}|$ grows with C , so the margin narrows; the number of support vectors falls; and every support vector with $0 < \alpha_i < C$ sits exactly on the margin, as the middle case of (8) requires.

4.3 Kernels (§2.3)

```
1 phi <- function(Z) cbind(1, sqrt(2)*Z[, 1], sqrt(2)*Z[, 2],
2                          Z[, 1]^2, Z[, 2]^2, sqrt(2)*Z[, 1]*Z[, 2])
3 Kq <- (1 + X2 %*% t(X2))^2
4 c(gram_from_map_vs_kernel = max(abs(phi(X2) %*% t(phi(X2)) - Kq)),
5   duals_agree_to = max(abs(dualfit(Kq, y2, 1) -
6                           dualfit(phi(X2) %*% t(phi(X2)), y2, 1))),
7   smallest_eigenvalue_of_K = min(eigen(Kq, symmetric = TRUE,
8                                       only.values = TRUE)$values))
```

```
## gram_from_map_vs_kernel      duals_agree_to
##      1.421e-14      4.148e-07
```

```
## smallest_eigenvalue_of_K  
## -7.953e-14
```

The Gram matrix is positive semi-definite, as Proposition [14.12](#) requires of any kernel arising from a feature map, so the dual stays a concave maximisation.

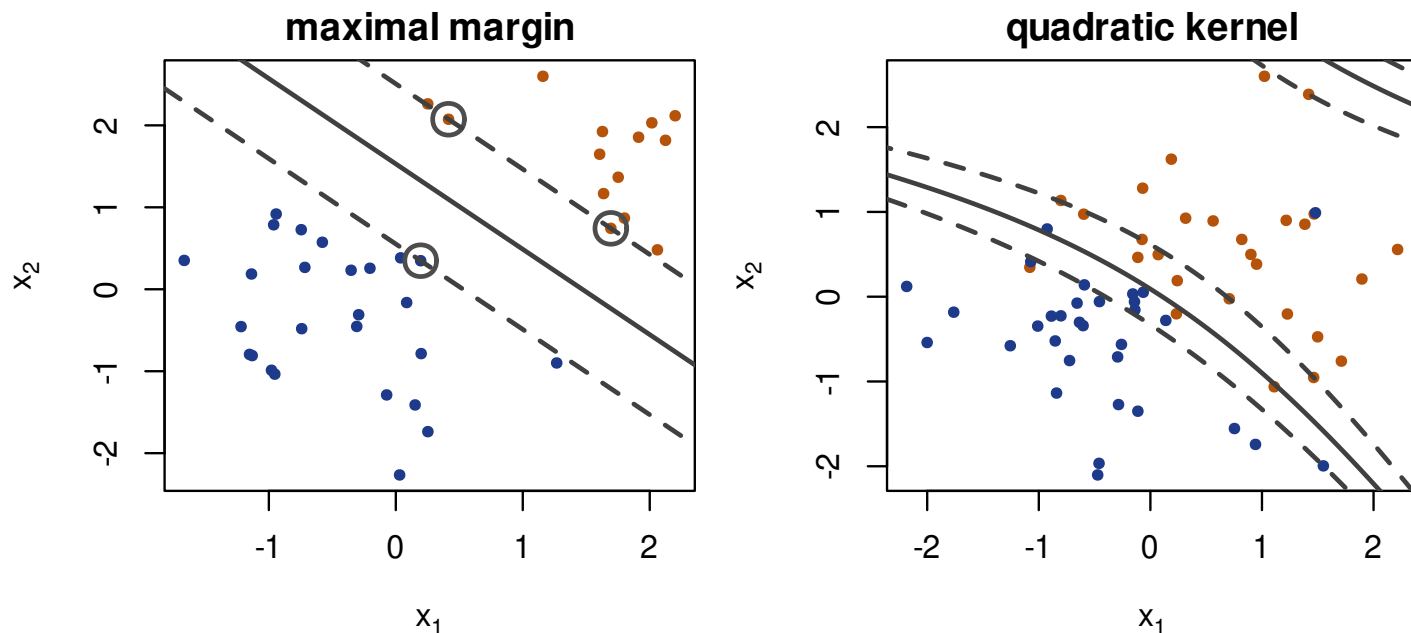


Figure 14.1: Left: the maximal margin hyperplane with its two margin boundaries and the three support vectors circled; by Corollary 14.7 moving any other point, so long as it stays outside the slab, changes nothing. Right: the same construction with $K(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}'\mathbf{z})^2$ on non-separable data, where the boundary is a conic because the machine is linear in the six features of Example 14.1.

4.4 Backpropagation (§3.2)

```

1  set.seed(2); nn <- 30; pp <- 3; H <- 5
2  Xn <- matrix(rnorm(nn*pp), nn, pp); yn <- rnorm(nn)
3  unpack <- function(th) list(W1 = matrix(th[1:(H*pp)], H, pp),
4    c1 = th[H*pp + 1:H], w2 = th[H*pp + H + 1:H], c2 = th[H*pp + 2*H + 1])
5  sig <- function(z) 1/(1 + exp(-z))
6  fwd <- function(th) { q <- unpack(th)
7    Z <- Xn %*% t(q$W1) + rep(q$c1, each = nn); A <- sig(Z)
8    list(A = A, f = drop(A %*% q$w2) + q$c2) }
9  loss <- function(th) mean((yn - fwd(th)$f)^2)/2
10 grad <- function(th) { q <- unpack(th); o <- fwd(th)
11   d2 <- (o$f - yn)/nn
12   D1 <- outer(d2, q$w2) * o$A * (1 - o$A)
13   c(as.vector(t(D1) %*% Xn), colSums(D1), drop(t(o$A) %*% d2), sum(d2)) }
14 th <- rnorm(H*pp + 2*H + 1, sd = 0.5)
15 num <- sapply(seq_along(th), function(j) { e <- rep(0, length(th))
16   e[j] <- 1e-6; (loss(th + e) - loss(th - e))/2e-6 })
17 c(parameters = length(th), largest_absolute_gap = max(abs(grad(th) - num)),
18   largest_relative_gap = max(abs(grad(th) - num)/(abs(num) + 1e-10)))

```

```
##           parameters largest_absolute_gap
##           2.600e+01      7.266e-11
## largest_relative_gap
##           3.072e-07
```

The derivative of the logistic activation is $g' = g(1 - g)$, so `o$A * (1 - o$A)` appears in place of $g'(\mathbf{Z})$ in (11).

```
1 set.seed(4); Hr <- 6
2 a1 <- rnorm(Hr); w1 <- rnorm(Hr); w2 <- rnorm(Hr); c2 <- rnorm(1)
3 fr1 <- function(x) c2 + sum(w2*pmax(0, w1*x + a1))
4 brk <- sort(-a1/w1)
5 mid <- c(brk[1] - 1, (brk[-Hr] + brk[-1])/2, brk[Hr] + 1)
6 slope <- sapply(mid, function(z) (fr1(z + 1e-6) - fr1(z - 1e-6))/2e-6)
7 c(units = Hr, breakpoints = length(unique(round(brk, 8))),
8   pieces = length(mid), distinct_slopes = length(unique(round(slope, 6))),
9   upper_bound_on_pieces = Hr + 1)
```

```
##           units           breakpoints
##           6             6
##           pieces        distinct_slopes
##           7             7
```

```
## upper_bound_on_pieces
## 7
```

4.5 Convex against non-convex (§3.4)

```
1 suppressMessages(library(nnet))
2 hinge <- function(v, C) { ww <- v[1:2]; bb <- v[3]
3   sum(pmax(0, 1 - y2*(X2 %*% ww + bb))) + sum(ww^2)/(2*C) }
4 svmruns <- sapply(1:10, function(s) { set.seed(s)
5   optim(rnorm(3, sd = 3), hinge, C = 1, method = "BFGS",
6     control = list(reltol = 1e-14, maxit = 5000))$value })
7 nnruns <- sapply(1:10, function(s) { set.seed(s)
8   nnet(Xn, yn, size = 5, linout = TRUE, trace = FALSE, maxit = 400)$value })
9 rbind(hinge_objective = round(svmruns, 5), nnet_objective = round(nnruns, 5))
```

```
##           [,1]  [,2]  [,3]  [,4]  [,5]  [,6]
## hinge_objective 18.604 18.605 18.605 18.60 18.605 18.605
## nnet_objective  7.556 7.073 4.048 4.24 3.581 4.703
##           [,7]  [,8]  [,9] [,10]
## hinge_objective 18.605 18.616 18.605 18.62
## nnet_objective  6.777 2.549 9.133 1.46
```

```
1 c(hinge_spread = diff(range(svmruns)), nnet_spread = diff(range(nnruns)),  
2   hinge_relative = diff(range(svmruns))/min(svmruns),  
3   nnet_relative = diff(range(nnruns))/min(nnruns))
```

```
##   hinge_spread      nnet_spread hinge_relative  nnet_relative  
##      0.0162020        7.6729238      0.0008709      5.2541112
```

Both are minimised from ten random starting points: the soft-margin objective in its unconstrained hinge form of Proposition 14.10, and the network. The first returns the same value from every start, to within one part in a thousand, which is the optimiser's tolerance on an objective that is convex but not differentiable at the kinks. The second returns a range of a factor of six. That is the difference between a convex programme, whose solution Theorem 14.5 makes unique, and a non-convex one.

5 Pitfalls

5.1 Not standardising before a kernel

Every kernel in §2.3 is a function of inner products or of distances, both of which are computed in whatever units the columns carry, exactly as in Unit 13.

```

1 set.seed(8); n3 <- 200
2 Z <- cbind(rnorm(n3), 100*rnorm(n3))
3 lab <- factor(ifelse(Z[, 1] + Z[, 2]/100 > 0, "a", "b"))
4 err <- function(sc) { i <- 1:100
5   m <- svm(Z[i, ], lab[i], kernel = "radial", cost = 1, scale = sc)
6   mean(predict(m, Z[-i, ]) != lab[-i]) }
7 c(unscaled = err(FALSE), scaled = err(TRUE))

```

```

## unscaled    scaled
##      0.37      0.04

```

Diagnostic. A radial or polynomial kernel whose fit changes when a column is re-expressed in different units. **Fix.** Standardise. `svm` does so by default, so the failure here had to be provoked with `scale = FALSE`: the second column has a hundred times the spread of the first, so $|\mathbf{x} - \mathbf{z}|^2$ is essentially its squared difference alone and the first column is invisible to the kernel.

5.2 Reading C as a bound on errors

```

1 t(sapply(c(0.01, 0.1, 1, 100), function(C) {
2   a <- dualfit(K2, y2, C); ww <- drop(t(X2) %*% (a*y2))

```

```

3 free <- a > 1e-7 & a < C - 1e-7
4 bb <- if (any(free)) mean(y2[free] - X2[free, , drop = FALSE] %*% ww) else 0
5 c(C = C, support_vectors = sum(a > 1e-7),
6   training_errors = sum(drop(y2*(X2 %*% ww + bb)) < 0),
7   margin = 2/sqrt(sum(ww^2))) })))

```

##		C	support_vectors	training_errors	margin
##	[1,]	0.01	50	26	6.0616
##	[2,]	0.10	35	4	1.6975
##	[3,]	1.00	22	4	0.8811
##	[4,]	100.00	17	2	0.6770

Diagnostic. A sentence of the form “ C controls how many points may be misclassified”. **Fix.** By Proposition 14.10 the role of C is that of $1/(2\lambda)$ in a penalised fit, so small C means heavy shrinkage of $\mathbf{w}$, a wide margin and many support vectors, and large C the reverse. It bounds nothing: C multiplies the total hinge loss and no individual constraint, so no value of C promises any particular number of mistakes. Here fifty of the sixty observations are support vectors at $C = 0.01$, where the margin is so wide that twenty-six fall on the wrong side, and seventeen at $C = 100$ with two wrong. The count moves with C and is nowhere equal to it.

5.3 Counting support vectors as a model-size measure

Diagnostic. A claim that a machine with few support vectors is simpler, used to choose C . **Fix.** §4.2 shows the count falling as C rises, which is the direction opposite to complexity: large C narrows the margin and fits harder. The count is a property of the geometry, not a penalty, and C is chosen by cross-validation on the prediction error like any other tuning parameter.

5.4 Reporting one run of a network

```

1 suppressMessages(library(nnet))
2 set.seed(11)
3 v <- sapply(1:20, function(s) { set.seed(s)
4   nnet(Xn, yn, size = 5, linout = TRUE, trace = FALSE, maxit = 400)$value })
5 c(best = min(v), worst = max(v), median = median(v),
6   distinct_to_six_places = length(unique(round(v, 6))))

```

```

##                best                worst
##                1.234                13.589
##      median distinct_to_six_places
##                4.323                20.000

```

Diagnostic. A network reported without saying how many starts were tried. **Fix.** Proposition 14.16

guarantees at least $H! = 120$ global minima here and says nothing about local ones, and gradient descent returns the basin it started in. Run from several starts, keep the best by a held-out criterion, and report the number of starts, exactly as Unit 13 requires of K -means, and for the same reason.

5.5 Trusting a hand-derived gradient

Diagnostic. A backpropagation implementation that trains without error and converges slowly to something plausible. **Fix.** Check the gradient against central differences before using it, as §4.4 does. The first version of that section's **grad** returned $\mathbf{X}'\mathbf{D}_1$ where (12) requires $\mathbf{D}_1'\mathbf{X}$, a transposition that leaves the dimensions of the parameter vector correct and every value wrong: the difference check reported a largest gap of 0.03 against 7×10^{-11} after the repair. Nothing in the training loop would have revealed it.

6 Exercises

Exercise 14.1. A separating hyperplane has $\mathbf{w} = (3, 4)'$ and $b = -10$. Give the geometric margin of the point $\mathbf{x} = (2, 3)'$ with $y = +1$, and the width of the slab if this is the canonical solution of (2).

Solution. $\mathbf{w}'\mathbf{x} + b = 6 + 12 - 10 = 8$, so the functional margin is 8 and by Proposition 14.2 the geometric margin is $8/|\mathbf{w}| = 8/5 = 1.6$.

If $(\mathbf{w}, b)$ solves (2) then the smallest functional margin over the data is 1, so the smallest geometric margin is $1/5 = 0.2$ and the slab is $2/|\mathbf{w}| = 0.4$ wide. The point above, at 1.6, is then far outside the slab and is not a support vector; by Corollary 14.7 its multiplier is zero and moving it a little changes nothing. ■

Exercise 14.2. Why is the dual (3) solved rather than the primal (2)? Give two reasons and say what guarantees the two give the same answer.

Solution. First, the dual has n variables and one equality constraint whatever p is, so it is the cheaper problem when $p \gg n$; the primal has $p + 1$. Second, and decisively, the dual involves the data only through the inner products $\mathbf{x}_i'\mathbf{x}_j$, so a kernel may be substituted (Proposition 14.12) and the machine fitted in a feature space whose dimension may be infinite, where the primal cannot even be written down.

They agree because strong duality holds: the constraints of (2) are affine and the problem is feasible, so the prerequisites' Slater condition applies and the optimal values coincide, with $\mathbf{w}^* = \sum_i \alpha_i^* y_i \mathbf{x}_i$

recovering the primal solution from the dual one. ■

Exercise 14.3. A fitted machine reports $\alpha_1 = 0$, $\alpha_2 = 0.4$ and $\alpha_3 = C$. What does each say about the corresponding observation?

Solution. By (8). Observation 1 has $\alpha = 0$, so its margin constraint is slack: it lies strictly outside the slab, is classified correctly, and contributes nothing to $\mathbf{w}^*$. Observation 2 has $0 < \alpha < C$, so it lies exactly on a margin boundary with functional margin 1; these are the observations from which b^* is recovered. Observation 3 has $\alpha = C$, so its slack is positive: it lies inside the slab, and it may or may not be on the wrong side of the hyperplane.

Only the second and third kinds enter (6). That the first kind can be moved freely, so long as it stays outside the slab, is complementary slackness and not an approximation. ■

Exercise 14.4. Show that $K(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}'\mathbf{z})^2$ on $\mathbb{R}^2$ is an inner product of explicit features, and say why that makes the dual a convex problem.

Solution. Expanding, $(1 + x_1z_1 + x_2z_2)^2 = 1 + 2x_1z_1 + 2x_2z_2 + x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2$, which is $\varphi(\mathbf{x})'\varphi(\mathbf{z})$ for $\varphi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)'$ as in Example 14.1.

By Proposition 14.12 any Gram matrix of a feature map is positive semi-definite, because $\mathbf{c}'\mathbf{K}\mathbf{c} = |\sum_i c_i \varphi(\mathbf{x}_i)|^2$. The dual's quadratic form is then negative semi-definite, so the objective is concave and the feasible set is an intersection of a hyperplane with a box, hence convex: a convex programme with a unique optimal value, to which the prerequisites' duality theory applies unchanged. ■

Exercise 14.5. Write the soft-margin problem as an unconstrained penalised loss and compare it with the lasso and ridge problems of Unit 6.

Solution. Proposition 14.10: minimise $\sum_i \max\{0, 1 - y_i(\mathbf{w}'\mathbf{x}_i + b)\} + |\mathbf{w}|^2/(2C)$, obtained by noting that the optimal slack is $\xi_i = \max\{0, 1 - \hat{\gamma}_i\}$.

The shape is Unit 6's: a loss plus a penalty, with $1/(2C)$ playing the part of λ . The differences are that the loss is the hinge rather than squared error, so it is convex but not differentiable at $\hat{\gamma}_i = 1$ and is exactly zero for confidently correct points, which is where the sparsity in the α_i comes from; and that the penalty is ℓ^2 , so it shrinks $\mathbf{w}$ without setting coordinates to zero. Sparsity here is in the observations, not in the coefficients. ■

Exercise 14.6. A network has $p = 4$ inputs, $H = 10$ hidden units and one output. How many parameters, and why is backpropagation preferred to computing each partial derivative by a difference quotient?

Solution. $Hp + 2H + 1 = 40 + 20 + 1 = 61$.

A central difference costs two forward passes per parameter, so 122 forward passes for one gradient. Theorem 14.14 obtains all 61 derivatives from one forward pass and one backward pass of the same cost, because the four identities are matrix products of quantities already computed. The saving grows with the number of parameters, which is the whole reason large networks are trainable at all.

Difference quotients are still worth computing once, on a small network, to check the implementation: §5.5 records a transposition that only that check would find. ■

Exercise 14.7. How many linear pieces can a one-dimensional network with rectified units and $H = 4$ hidden units produce, and what does that say about the class of functions it fits?

Solution. By Theorem 14.15(i) at most $H + 1 = 5$ pieces, with breakpoints at the four points $-c_{1h}/w_{1h}$. Part (ii) says the bound is attained and that every continuous piecewise-linear function with four breakpoints arises this way.

So the class is piecewise-linear regression with four free knots, and the network's advantage over a spline with fixed knots is that the knots are fitted. Compare Unit 11: a regression tree with four splits is piecewise **constant** on five intervals, so it cannot represent a slope at all, which is the staircase error that unit measures. ■

Exercise 14.8. Two analysts fit the same support vector machine and get identical answers; two analysts fit the same network and do not. Explain both.

Solution. The support vector machine solves (2), a strictly convex objective over a closed convex feasible set, which by Theorem 14.5 has exactly one solution. Any algorithm that reaches optimality reaches the same point, and the dual's optimal value is the primal's by strong duality, so even the diagnostic quantities agree.

The network minimises L in Definition 14.13, which by Proposition 14.16 has at least $H!$ global minima, since permuting the hidden units leaves f and hence L unchanged. A function with two

minima and a larger value between them is not convex, so gradient descent has basins and returns the one it started in. §4.5 runs both ten times: one number and a range. ■

Exercise 14.9. In R: (a) solve the dual quadratic programme and check $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $|\mathbf{w}|^2 = \sum_i \alpha_i$ against a package; (b) verify complementary slackness; (c) verify a backpropagation gradient against central differences.

Solution. (a) and (b) are §4.1, where the functional margin at every support vector is 1 to seven decimal places and the two expressions for the margin width agree; (c) is §4.4, agreeing to 10^{-10} . The point to watch in (a) is the sign convention: **e1071** orders the factor levels its own way, so its $\mathbf{w}$ and b are the negatives of the ones derived here and the sums rather than the differences are what vanish. The hyperplane and the classifier are the same object; only the labelling of the two sides differs. ■